

山东大学



网络安全创新创业实践

——对称密码算法的软件实现及优化

7.24 作业 3

姓名	学号	班级
钟晴	202322460127	密码 2 班
宋坤鹏	202322460130	密码 1 班
赵思鹏	202300460075	密码 1 班
田学琛	202300460064	密码 1 班

山东大学网络空间安全学院

目录

1 实验目的	1
2 实验原理及基础知识	1
2.1 对称密码概述	1
2.2 软件实现的基本方法	1
2.3 SIMD 指令集与数据集并行	2
2.4 专用加密指令与位操作扩展	2
2.5 工作模式原理优化	2
3 SM4 算法实现及优化	3
3.1 SM4 算法简介	3
3.2 基础实现 (Base)	3
3.3 T-Table 优化	4
3.4 SIMD 向量化 S 盒优化 (SSSE3 / AES-NI)	5
3.5 工作模式实现优化	5
3.6 实验结果	6
3.7 SM4 优化小结	7
4 AES 算法实现及优化	7
4.1 AES 算法简介	7
4.2 基础实现 (Base)	8
4.3 T-Table 优化	8
4.4 Shuffle 优化: ShiftRows 加速的尝试与反思	9
4.5 AES-NI 硬件加速	10
4.6 AVX2 并行优化	10
4.7 工作模式实现优化	11
4.8 实验结果	11
4.9 优化小结	13
5 GIFT-64 算法实现及优化	13
5.1 GIFT-64 算法简介	13
5.2 基础实现 (Base)	14
5.3 T-Table 优化	14
5.4 SIMD 向量化 S 盒尝试 (SSSE3 / AES-NI / AVX2)	15
5.5 工作模式实现优化	15
5.6 实验结果	16
5.7 优化小结与展望	17

6	TWINE 算法实现及优化	18
6.1	TWINE 算法简介	18
6.2	基础实现 (Base)	19
6.3	T-Table 优化	19
6.4	SSSE3 向量化 S 盒优化	20
6.5	AVX2 256 位并行尝试	21
6.6	BMI2 比特置换优化	22
6.7	工作模式实现优化	22
6.8	实验结果	23
6.9	优化小结与启示	25
7	总结	26
7.1	各算法最优方案及加速比汇总	26
7.2	优化技术的适用性分析	26
7.3	工作模式对性能的影响	27
7.4	整体结论与展望	27

1 实验目的

作业 3: 对称密码算法的软件实现从基本实现出发优化对称密码 (SM4 / AES / GIFT / TWINE) 的软件执行效率, 至少应该覆盖 T-table、shuffle 以及最新的指令集中的两种方法基于加密解密软件实现, 做 CTR / GCM / XTS 工作模式的软件优化实现

本实验旨在深入研究对称密码算法在通用处理器上的软件实现技术, 以 AES-128、SM4、GIFT-64 和 TWINE 四种典型分组密码为对象, 从基本实现出发, 逐步引入多种优化手段, 探索如何充分利用现代 CPU 的指令集特性 (如 SIMD、AES-NI、BMI2 等) 和微架构特点 (如流水线、多发射、缓存层次) 来显著提升加密/解密吞吐率。同时, 将优化后的分组密码核应用于 CTR、GCM 和 XTS 三种常用工作模式, 评估不同实现策略在实际场景下的性能表现。

2 实验原理及基础知识

2.1 对称密码概述

对称密码算法是加密和解密使用相同密钥的密码体制, 其核心操作通常包括非线性替换 (S 盒)、线性扩散 (置换、矩阵乘法、移位) 和密钥加 (异或)。本实验涉及四种典型分组密码:

(1)AES-128: 美国国家标准, 分组 128 位, 密钥 128 位, 10 轮。每轮由字节替换 (S 盒)、行移位、列混合和轮密钥加组成, 具有优秀的软件和硬件实现性能。

(2)SM4: 中国商用密码标准, 分组 128 位, 密钥 128 位, 32 轮。采用非平衡 Feistel 结构, 轮函数包含 S 盒、非线性变换 和线性变换 L, 适合 8 位/32 位平台。

(3)GIFT-64: 轻量级密码, 分组 64 位, 密钥 128 位, 28 轮。基于 SPN 结构, S 盒 4×4 , 线性层为比特置换, 硬件面积小, 适合资源受限环境。

(4)TWINE: 轻量级密码, 分组 64 位, 密钥 80 位, 36 轮。采用广义 Feistel 结构, 每轮使用 4 位 S 盒和 8 位洗牌, 设计简洁, 兼顾软硬件实现。

2.2 软件实现的基本方法

2.2.1 朴素实现

直接按算法规范描述进行逐轮运算, 使用数组/循环模拟所有操作。优点是正确性易于验证, 但性能较差, 因为大量内存访问、循环分支和逐字节处理无法发挥现代 CPU 的并行能力。

2.2.2 T-table 实现

将 S 盒替换与线性变换 (如 AES 的列混合、SM4 的 L 变换) 预先合并为查找表。例如 SM4 可将 $T(x) = L(S(x))$ 拆分为 4 个 32 位表, 查表后异或得到轮函数输出。T-Table 以空间换时间, 减少了每轮中的字节提取和移位运算, 但表的大小需适配 L1 缓存 (通常 32KB), 过大表会导致缓存缺失。

2.2.3 循环展开

将固定轮数的循环完全展开为顺序代码，消除循环计数和跳转开销，同时为编译器提供更多指令调度机会。对于轮数较小的算法（如 GIFT 28 轮、TWINE 36 轮），全展开能显著提升性能。

2.2.4 寄存器优化

尽量将状态变量保存在通用寄存器中，避免频繁的栈内存读写。现代 x86-64 有 16 个通用寄存器，编译器可自动分配，但需注意寄存器溢出。手动使用 `register` 关键字或内联汇编可进一步控制寄存器使用。

2.3 SIMD 指令集与数据集并行

单指令多数据流（SIMD）允许一条指令同时处理多个数据元素，是提高对称密码吞吐率的关键技术。(1)`shuffle:128` 位字节洗牌指令，可按 16 个独立索引从 16 字节向量中选取字节，常用于 4 位/8 位 S 盒的并行查表，配合掩码可实现 4 路或 8 路并行。

(2)`AVX2`: 将向量宽度扩展到 256 位，支持整数操作（加、异或、移位等），可一次处理两个 128 位块或 8 个 32 位字，提升 CTR/GCM 等模式下的批量异或效率。

利用 SIMD 进行并行 CTR 等多种工作方式加密时，预计算多个计数器的密钥流，然后一次异或多块明文，可大幅减少每字节的指令数。

2.4 专用加密指令与位操作扩展

(1)`AES-NI`: Intel/AMD 提供的硬件加速指令，单条指令完成 AES 的一轮 (`SubBytes`、`ShiftRows`、`MixColumns`、`AddRoundKey`)，使 AES 加密达到接近内存带宽的速率。本实验中的 AES-NI 实现直接调用这些指令。(2)`BMI2`: 位并行提取/沉积指令，可高效实现比特级置换操作。对 TWINE 等轻量级密码的线性层（洗牌）有显著加速效果，但不同微架构支持程度不同。

2.5 工作模式原理优化

分组密码通常工作于特定模式以加密任意长度数据。(1)CTR（计数器模式）：将计数器初始值 (IV) 逐块递增，用分组密码加密得到密钥流，然后与明文异或。CTR 本质可完全并行，适合 SIMD 批量处理。本实验对 CTR 采用 4 路、8 路并行加密，并合并计数器更新。(2)GCM（伽罗瓦/计数器模式）：在 CTR 基础上增加认证功能，使用 GHASH 多项式哈希。本实验主要关注加密部分的 CTR 核心，认证部分未全部实现，但提供了 GCM 风格的 CTR 初始化（将 IV 填充为 12 字节 + 计数器）。(3)XTS（基于 XEX 的可调分组密码模式）：主要用于磁盘加密，每个数据块使用不同 tweak（由 IV 和块索引派生）。XTS 的瓶颈在于 tweak 更新（有限域乘 2）和两次分组加密。优化时可利用双路并行：同时处理两个连续块，预计算 tweak 的乘 2 和乘 4，减少等待。优化要点：在多路并行中，尽量合并 tweak 更新操作，或预先计算多个 tweak 值，减少 GF 乘法开销。

3 SM4 算法实现及优化

3.1 SM4 算法简介

SM4 是中国国家密码管理局公布的分组密码标准，分组长度为 128 位（4 个 32 位字），密钥长度 128 位，迭代轮数 32 轮。其结构为**非平衡 Feistel 网络**，加密过程可描述为：

- 将 128 位明文分为 (X_0, X_1, X_2, X_3) （每个 32 位）。
- 对于第 i 轮 $(0 \leq i \leq 31)$ ：

$$X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus RK_i)$$

其中 T 为合成变换： $T(A) = L(\tau(A))$ 。 τ 由 4 个并行的 8 位 S 盒构成：

$$(B_0, B_1, B_2, B_3) = (Sbox(A_0), Sbox(A_1), Sbox(A_2), Sbox(A_3))$$

L 为线性变换：

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

- 32 轮后，输出 $(X_{35}, X_{34}, X_{33}, X_{32})$ 并做字节反序得到密文。

密钥扩展类似，使用相同的 T 变换和系统参数 FK 、 CK 。

SM4 的软件实现挑战在于：

1. 每轮需 4 次 S 盒查表和 4 次循环移位，指令数较多。
2. Feistel 结构导致数据依赖链较长，不利于指令级并行。
3. 密钥扩展与加密共享同一 T 函数，可重用优化成果。

3.2 基础实现 (Base)

基础实现严格遵循算法定义，采用循环和数组存储状态。典型代码片段如下：

```
1 void sm4_encrypt_base(const uint8_t in[16], uint8_t out[16], const uint32_t rk[32]) {
2     uint32_t X[4];
3     for (int i = 0; i < 4; i++) {
4         X[i] = (in[4*i]<<24) | (in[4*i+1]<<16) |
5             (in[4*i+2]<<8) | in[4*i+3];
6     }
7     for (int i = 0; i < 32; i++) {
8         uint32_t x4 = X[0] ^ sm4_T(X[1] ^ X[2] ^ X[3] ^ rk[i]);
9         X[0] = X[1]; X[1] = X[2]; X[2] = X[3]; X[3] = x4;
10    }
11    // 反序输出
12    for (int i = 0; i < 4; i++) {
13        uint32_t val = X[3 - i];
```

```
14     out[4*i] = val >> 24; out[4*i+1] = val >> 16;  
15     out[4*i+2] = val >> 8; out[4*i+3] = val;  
16 }  
17 }
```

此实现结构清晰，但存在以下性能瓶颈：

- 每轮都需调用 `sm4_T`，包含多次 S 盒查表和循环移位，循环控制开销不可忽视。
- 状态数组 X 存储在内存中，寄存器使用效率低。
- 32 轮循环分支影响流水线。

基础实现仅作为正确性参考，实际性能较低（实验中 CTR 模式约 117 MB/s）。

3.3 T-Table 优化

3.3.1 T 盒构造原理

合成变换 $T(A) = L(\tau(A))$ 中， A 为 32 位字，可拆分为 4 个字节 a_0, a_1, a_2, a_3 。由于 L 是线性变换，可改写为：

$$T(A) = L(S(a_0)) \oplus L(S(a_1) \ll 8) \oplus L(S(a_2) \ll 16) \oplus L(S(a_3) \ll 24)$$

预计算 4 个 T 盒：

$$T_0[x] = L(S(x) \ll 24)$$

$$T_1[x] = L(S(x) \ll 16)$$

$$T_2[x] = L(S(x) \ll 8)$$

$$T_3[x] = L(S(x))$$

则轮函数可简化为：

```
1 uint32_t tmp = X1 ^ X2 ^ X3 ^ rk[i];  
2 uint32_t x4 = X0 ^ T0[tmp>>24] ^ T1[(tmp>>16)&0xff]  
3             ^ T2[(tmp>>8)&0xff] ^ T3[tmp&0xff];
```

这样每轮只需 4 次查表和 4 次异或，完全避免了循环移位和字节拆分的开销。

3.3.2 实现细节

本实验采用标准 4 表方案，每个表 $256 \times 4 = 1024$ 字节，共 4 KB，可完全放入 L1 缓存。同时将 32 轮循环完全展开，消除了循环控制指令，并尽量将状态 $X_0 \sim X_3$ 保持在寄存器中，进一步减少内存访问。

3.3.3 性能收益

T-Table 版本的 CTR 模式吞吐率达到 185 MB/s，相比 Base 的 117 MB/s 提升约 58

3.4 SIMD 向量化 S 盒优化 (SSSE3 / AES-NI)

与常见的仅将 T-Table 并行化的做法不同, 本实验对 SSSE3 和 AES-NI 方案实现了真正向量化的 S 盒替换, 以充分利用现代 SIMD 指令集的并行能力。

3.4.1 SSSE3 方案: 复合域分解与 pshufb 查表

SM4 的 S 盒定义在 $GF(2)$ 上, 不可约多项式为 $x^8 + x^7 + x^6 + x^5 + x^4 + x^2 + 1$ 。利用有限域同构映射将 $GF(2)$ 分解为 $GF(2)^2$, 使得每个字节可表示为高、低两个 4 位半字节。基于此, 预计算 16 个 16 字节表, 每个表对应高半字节的 16 种取值, 表内为对应的 S 盒输出。在每轮中, 将输入字节拆分为高、低 4 位, 使用 `_mm_shuffle_epi8` 根据低 4 位索引对应高 4 位所在的表, 一次性完成 16 个字节的 S 盒替换。该过程仅需若干条 shuffle 和逻辑指令, 避免了标量查表的循环开销。

3.4.2 AES-NI 方案: 仿射映射与硬件 S 盒指令

AES-NI 指令集中的 `_mm_aesenclast_si128` 可以直接对 16 字节执行 AES 的最后一轮 (Sub-Bytes + ShiftRows + AddRoundKey)。为实现 SM4 的 S 盒, 我们预先计算两个仿射映射表 ($SM4 \rightarrow AES$ 和 $AES \rightarrow SM4$), 同样利用 `_mm_shuffle_epi8` 进行字节级映射。在每轮加密中, 先将 SM4 状态字节映射到 AES 域, 调用 `_mm_aesenclast_si128` 获得 AES S 盒输出, 再逆映射回 SM4 域。此方法将 S 盒替换完全硬件化, 极大地减少了指令数。

3.4.3 AVX2 方案: 256 位并行 XOR

对于 AVX2 方案, 加密核心仍使用标量 T-Table, 但利用 `_mm256_xor_si256` 将两个 128 位块的异或操作合并为一条 256 位指令, 从而在模式层 (CTR/GCM) 进一步降低开销。

3.5 工作模式实现优化

3.5.1 CTR 模式

所有优化方案均实现了多路并行 CTR 模式:

- **SSSE3**: 4 路并行, 使用 128 位 XMM 寄存器同时处理 4 个计数器块。
- **AES-NI**: 8 路并行, 同样使用 128 位寄存器, 但组大小增至 8。
- **AVX2**: 8 路并行, 并利用 256 位 XOR 聚合两个块的操作。

各方案均采用连续计数器增量 (4 或 8) 以减少循环内更新开销。

3.5.2 GCM 模式

GCM 的加密部分与 CTR 完全相同, 仅初始计数器由 IV 和固定字段构建。因此本实验直接复用 CTR 并行函数, 性能与 CTR 基本一致。

3.5.3 XTS 模式

XTS 的 tweak 更新 ($GF(2^{12})$ 下的乘以 2) 具有串行依赖, 难以向量化, 因此所有优化方案在 XTS 下均退化为单块 T-Table 加密, 并行收益有限。

3.6 实验结果

本次实验在 Intel Core i7-13700 (VMware, Ubuntu 20.04) 平台上运行, 测试数据量为 100 MB, 每种方案重复 10 次取平均耗时, 并计算吞吐率。所有方案均通过正确性验证 (与 Base 密文一致)。性能数据汇总于表 1。

表 1: SM4 各模式性能数据 (100MB $\times$ 10 次)

实现方案	CTR		GCM		XTS	
	耗时 (s)	吞吐 (MB/s)	耗时 (s)	吞吐 (MB/s)	耗时 (s)	吞吐 (MB/s)
Base	8.526	117.29	8.520	117.37	9.296	107.58
T-Table	5.416	184.65	5.396	185.32	6.071	164.73
SSSE3	4.885	204.71	4.870	205.34	6.087	164.28
AES-NI	4.917	203.37	4.888	204.58	6.133	163.05
AVX2	4.931	202.82	4.900	204.08	6.143	162.79

```
newl@ubuntu:~$ ./sm4
```

SM4 多方案 & 多模式 完整性能对比测试					
CTR 模式 (100MB $\times$ 10次)					
CTR	Base		8.526 秒		117.29 MB/s
CTR	T-Table		5.416 秒		184.65 MB/s
CTR	SSSE3		4.885 秒		204.71 MB/s
CTR	AES-NI		4.917 秒		203.37 MB/s
CTR	AVX2		4.931 秒		202.82 MB/s
GCM 模式 (100MB $\times$ 10次)					
GCM	Base		8.520 秒		117.37 MB/s
GCM	T-Table		5.396 秒		185.32 MB/s
GCM	SSSE3		4.870 秒		205.34 MB/s
GCM	AES-NI		4.888 秒		204.58 MB/s
GCM	AVX2		4.900 秒		204.08 MB/s
XTS 模式 (100MB $\times$ 10次)					
XTS	Base		9.296 秒		107.58 MB/s
XTS	T-Table		6.071 秒		164.73 MB/s
XTS	SSSE3		6.087 秒		164.28 MB/s
XTS	AES-NI		6.133 秒		163.05 MB/s
XTS	AVX2		6.143 秒		162.79 MB/s
测试完成					

图 1: sm4 优化效能对比

分析结果可见：

- **SSSE3、AES-NI 和 AVX2** 在 CTR/GCM 模式下均达到约 203–205 MB/s，相比 Base 提升约 74
- **XTS 模式**下，所有优化方案仅能利用 T-Table 单块加密，吞吐率约 163 MB/s，较 Base 提升约 52
- **AES-NI** 方案通过仿射映射和硬件 S 盒指令获得了与 SSSE3 几乎相同的性能，表明该优化是有效的，且无需额外硬件支持（仅需 AES-NI 指令集）。

综上，SM4 通过 T-Table 和 SIMD 向量化 S 盒优化，在 x86-64 平台上获得了显著的性能提升，尤其适合 CTR/GCM 这类可并行的模式。而 XTS 模式因自身结构限制，性能提升主要依赖 T-Table，并行收益有限。

3.7 SM4 优化小结

通过从 Base → T-Table → SIMD 向量化 S 盒的逐步优化，SM4 在 CTR 模式下的性能从 117 MB/s 提升至 205 MB/s，提升约 **75%**，充分展示了预计算、数据级并行和现代 SIMD/硬件指令的威力。不同优化技术的适用性总结如表 2：

表 2: SM4 优化技术对比

优化技术	适用场景	性能收益	代价
T-Table	所有模式	高	4 KB 缓存占用
SSSE3 向量 S 盒	CTR/GCM	高	需 SSSE3 支持
AES-NI 仿射映射	CTR/GCM	高	需 AES-NI 支持
AVX2 256 位 XOR	CTR/GCM	中	需 AVX2 支持
循环展开	固定轮数	中	代码膨胀

对于实际应用，**SSSE3 向量 S 盒**方案在兼容性和性能间取得了良好平衡，适合大多数 x86-64 平台；而 AES-NI 方案则适用于支持 AES-NI 的 CPU，可进一步利用硬件加速。本实验结果为 SM4 在通用处理器上的高性能实现提供了有价值的参考。

4 AES 算法实现及优化

4.1 AES 算法简介

AES (Advanced Encryption Standard) 是美国国家标准与技术研究院 (NIST) 公布的分组密码标准，分组长度为 128 位，支持 128、192 和 256 位密钥。本实验采用 **AES-128**，迭代轮数为 10 轮。其结构为**替代-置换网络 (SPN)**，每轮包含以下变换：

- **字节替代 (SubBytes)**：对状态矩阵的每个字节应用 S 盒非线性变换。
- **行移位 (ShiftRows)**：将状态矩阵的行按不同偏移量循环移位。

- 列混合 (MixColumns): 在 $GF(2^8)$
- 轮密钥加 (AddRoundKey): 将状态与轮密钥异或。

初始轮仅执行轮密钥加, 最后一轮省略列混合。密钥扩展算法生成 11 个轮密钥 (每个 128 位)。AES 的软件实现面临以下挑战:

1. S 盒为 8 位查表, 每轮需 16 次查表, 缓存压力较大。
2. 列混合涉及 $GF(2^8)$
2. 各轮之间存在数据依赖, 但 AES 的高度规则性使其适合 SIMD 和硬件指令加速。

4.2 基础实现 (Base)

基础实现严格遵循算法定义, 采用循环和数组操作。核心代码片段如下:

```
1 void aes_encrypt_base(const uint8_t in[16], uint8_t out[16], const uint8_t rk[176]) {
2     uint8_t s[16]; memcpy(s, in, 16);
3     AddRoundKey(s, rk); // 初始轮
4     for (int r = 1; r < 10; r++) {
5         SubBytes(s); // S盒查表
6         ShiftRows(s);
7         MixColumns(s);
8         AddRoundKey(s, rk + r*16);
9     }
10    SubBytes(s);
11    ShiftRows(s);
12    AddRoundKey(s, rk + 10*16);
13    memcpy(out, s, 16);
14 }
```

此实现每轮需进行 16 次 S 盒查表、4 次列混合 (每列独立) 和若干循环移位, 指令数较多, 且数据在内存中频繁读写。实验中 Base 版本的 CTR 模式吞吐率仅为约 78 MB/s, 作为正确性参考基准。

4.3 T-Table 优化

4.3.1 T 盒构造原理

AES 的 SubBytes 和 MixColumns 可以合并为查表操作。对于每个状态字节 $a_{i,j}$, 定义:

$$T_0[x] = \begin{pmatrix} 2 \cdot S[x] \\ S[x] \\ S[x] \\ 3 \cdot S[x] \end{pmatrix}, \quad T_1[x] = \begin{pmatrix} 3 \cdot S[x] \\ 2 \cdot S[x] \\ S[x] \\ S[x] \end{pmatrix}, \quad T_2[x] = \begin{pmatrix} S[x] \\ 3 \cdot S[x] \\ 2 \cdot S[x] \\ S[x] \end{pmatrix}, \quad T_3[x] = \begin{pmatrix} S[x] \\ S[x] \\ 3 \cdot S[x] \\ 2 \cdot S[x] \end{pmatrix}$$

则一轮变换可简化为 4 次 32 位查表和 4 次异或操作:

```
1 uint32_t t0 = T0[s0] ^ T1[s1] ^ T2[s2] ^ T3[s3];
```

这样将 SubBytes 和 MixColumns 合并，减少了指令数。

4.3.2 实现细节

本实验采用标准 4 表方案，每个表大小为 $256 \times 4 = 1024$ 字节，共 4 KB，完全放入 L1 缓存。同时将 10 轮循环完全展开，消除循环控制开销。

4.3.3 性能收益

T-Table 版本的 CTR 模式吞吐率达到 404 MB/s，相比 Base 的 78 MB/s 提升约 5.2 倍，证明预计算和表查找对 AES 非常有效。

4.4 Shuffle 优化：ShiftRows 加速的尝试与反思

4.4.1 优化原理

Shuffle 方案尝试利用 SSSE3 的 `_mm_shuffle_epi8` 指令加速 ShiftRows 操作。该指令可在单条指令内完成 16 个字节的任意重排，非常适合实现 AES 的行移位。预期通过向量化减少循环中的字节移动开销。

4.4.2 实现方式

在每轮加密中，使用预定义的 shuffle 掩码一次性完成 ShiftRows：

```
1 __m128i mask = _mm_set_epi8(3,0,1,2, 7,4,5,6, 11,8,9,10, 15,12,13,14);  
2 state = _mm_shuffle_epi8(state, mask);
```

SubBytes、MixColumns 和 AddRoundKey 仍使用软件实现。

4.4.3 性能结果与反思

实验结果出人意料：Shuffle 方案的 CTR 吞吐率仅为 69.57 MB/s，甚至低于 Base 的 77.92 MB/s，在所有方案中表现最差。深入分析原因如下：

- **ShiftRows 并非性能瓶颈**：在 AES 的每轮中，ShiftRows 仅涉及 12 次字节移动，开销远小于 SubBytes（16 次查表）和 MixColumns（ $\text{GF}(2^8)$ *ShiftRows*）。
- **额外的加载/存储开销**：使用 `_mm_loadu_si128` 和 `_mm_storeu_si128` 需要在内存和 XMM 寄存器之间传输数据，而纯软件实现中 ShiftRows 直接在数组上操作，编译器可以优化寄存器分配。
- **破坏了编译器的优化**：混合使用 SIMD 指令和标量操作会限制编译器的优化空间（如寄存器分配、指令调度），导致整体性能下降。
- **指令延迟**：SIMD 指令通常具有较高的延迟和较低的吞吐量，对于小规模数据（16 字节），指令开销可能超过其带来的并行收益。

这一结果清晰地表明：**优化应聚焦于真正的性能瓶颈**，而非盲目应用 SIMD。对于 AES，SubBytes 和 MixColumns 才是优化的重点，ShiftRows 的加速收益微乎其微。

4.5 AES-NI 硬件加速

4.5.1 AES-NI 指令集简介

从 Intel Westmere 架构开始，x86 处理器引入了 AES-NI 指令集，提供硬件加速的 AES 加密/解密操作。核心指令包括：

- `_mm_aesenc_si128`: 执行单轮 AES 加密 (SubBytes + ShiftRows + MixColumns + AddRoundKey)
- `_mm_aesenclast_si128`: 执行最后一轮 AES 加密 (SubBytes + ShiftRows + AddRoundKey)
- `_mm_aeskeygenassist_si128`: 辅助密钥扩展

4.5.2 实现方式

AES-NI 方案直接调用硬件指令实现单块加密：

```
1 void aes_encrypt_aesni(const uint8_t in[16], uint8_t out[16], const uint8_t rk[176]) {  
2     __m128i state = _mm_loadu_si128((const __m128i*)in);  
3     const __m128i *rk128 = (const __m128i*)rk;  
4     state = _mm_xor_si128(state, rk128[0]); // 初始轮密钥加  
5     for (int i = 1; i < 10; i++)  
6         state = _mm_aesenc_si128(state, rk128[i]); // 9轮  
7     state = _mm_aesenclast_si128(state, rk128[10]); // 最后一轮  
8     _mm_storeu_si128((__m128i*)out, state);  
9 }
```

该方案将 10 轮加密压缩为 10 条硬件指令，极大地减少了 CPU 周期。在 CTR/GCM 模式中，进一步采用 8 路并行（8 个块同时加密），充分压榨硬件性能。

4.5.3 性能收益

AES-NI 方案在 CTR 模式下达到 3437 MB/s，相比 Base 提升约 44 倍，相比 T-Table 提升约 8.5 倍。这充分证明了硬件指令对 AES 的加速效果是革命性的。

4.6 AVX2 并行优化

4.6.1 AVX2 特性

AVX2 扩展了 SIMD 寄存器宽度至 256 位，并引入了更多向量操作指令。本实验利用 `_mm256_xor_si256` 实现两个 128 位块的并行异或，减少向量操作指令数。

4.6.2 实现方式

在 CTR/GCM 模式中，8 个块的加密仍使用 AES-NI 单块加密函数，但在异或操作阶段，将两个 128 位块（共 32 字节）拼接到一个 256 位 YMM 寄存器中，一次性完成异或：

```
1 for (int k = 0; k < 8; k += 2) {  
2     __m256i pt256 = _mm256_loadu_si256((__m256i*)(in + (b+k)*16));  
3     __m256i ks256 = _mm256_load_si256((__m256i*)(ctr_buf[k]));  
4     pt256 = _mm256_xor_si256(pt256, ks256);  
5     _mm256_storeu_si256((__m256i*)(out + (b+k)*16), pt256);  
6 }
```

4.6.3 性能收益

AVX2 方案在 CTR 模式下达到 3558 MB/s, 略高于 AES-NI 的 3437 MB/s (提升约 3.5%)。256 位异或减少了指令数, 但由于内存带宽已成为瓶颈, 边际收益有限。

4.7 工作模式实现优化

4.7.1 CTR 模式

CTR 模式将计数器值加密生成密钥流, 然后与明文异或。本实验为不同实现提供了专用函数:

- Base 和 T-Table 使用单块加密, 计数器逐一递增。
- Shuffle 方案使用单块加密 (但使用了 shuffle 加速 ShiftRows)。
- AES-NI 使用 8 路并行: 预计算 8 个连续计数器值, 分别加密后与 8 个明文块异或。
- AVX2 同样使用 8 路并行, 但异或阶段采用 256 位操作。

4.7.2 GCM 模式

GCM 的加密部分与 CTR 相同, 仅初始计数器根据 IV 构建 (12 字节 IV + 4 字节计数器初始值)。本实验直接复用 CTR 的并行函数, 因此性能与 CTR 基本一致。

4.7.3 XTS 模式

XTS 模式为每个块生成独立的 tweak 值, 且 tweak 在 $GF(2^{128})$ 中 $2 \times tweak$

Base XTS

4.8 实验结果

本次实验在 Intel Core i7-13700 (VMware, Ubuntu 20.04) 平台上运行, 测试数据量 100 MB, 每种方案重复 10 次取平均耗时并计算吞吐率。所有实现的正确性均已通过 Base 密文验证。终端输出截图如图 2 所示。

```
newl@ubuntu:~$ ./aes
```

AES-128 多方案 & 多模式 完整性能对比测试			
===== CTR 模式 (100MB × 10次) =====			
CTR	Base	12.834 秒	77.92 MB/s
CTR	T-Table	2.474 秒	404.20 MB/s
CTR	Shuffle	14.373 秒	69.57 MB/s
CTR	AES-NI	0.291 秒	3437.25 MB/s
CTR	AVX2	0.281 秒	3558.17 MB/s
===== GCM 模式 (100MB × 10次) =====			
GCM	Base	13.179 秒	75.88 MB/s
GCM	T-Table	2.550 秒	392.19 MB/s
GCM	Shuffle	14.590 秒	68.54 MB/s
GCM	AES-NI	0.296 秒	3376.83 MB/s
GCM	AVX2	0.307 秒	3261.49 MB/s
===== XTS 模式 (100MB × 10次) =====			
XTS	Base	13.997 秒	71.45 MB/s
XTS	T-Table	3.233 秒	309.33 MB/s
XTS	Shuffle	15.702 秒	63.69 MB/s
XTS	AES-NI	1.439 秒	695.06 MB/s
XTS	AVX2	1.423 秒	702.50 MB/s
===== 测试完成 =====			

图 2: AES-128 多方案多模式性能测试结果

性能数据汇总于表 3。

表 3: AES-128 各模式性能数据 (100MB × 10 次)

实现方案	CTR		GCM		XTS	
	耗时 (s)	吞吐 (MB/s)	耗时 (s)	吞吐 (MB/s)	耗时 (s)	吞吐 (MB/s)
Base	12.834	77.92	13.179	75.88	13.997	71.45
T-Table	2.474	404.20	2.550	392.19	3.233	309.33
Shuffle	14.373	69.57	14.590	68.54	15.702	63.69
AES-NI	0.291	3437.25	0.296	3376.83	1.439	695.06
AVX2	0.281	3558.17	0.307	3261.49	1.423	702.50

分析结果可见：

- **正确性**：所有方案加密结果与 Base 完全一致。
- **T-Table 效果显著**：相比 Base 提升约 5.2 倍，证明了查表法的有效性。
- **Shuffle 优化失败**：性能甚至低于 Base，原因是优化了非瓶颈操作且引入了额外开销。
- **AES-NI 硬件加速效果惊人**：达到 3437 MB/s，比 Base 快 44 倍，是 AES 最优实现方案。
- **AVX2 略有提升**：在 AES-NI 基础上利用 256 位异或再提升约 3.5%，但边际收益递减。

- **XTS 模式限制并行性**：所有方案在 XTS 下性能大幅下降（约 700 MB/s），无法发挥 SIMD 优势。

4.9 优化小结

通过从纯软件到硬件指令，再到向量并行的逐步优化，AES 在 CTR 模式下的吞吐率从 78 MB/s 跃升至近 3.6 GB/s，提升了约 **45 倍**。不同优化技术的适用性总结如表 4。

表 4: AES 优化技术对比

优化技术	适用场景	性能收益	代价
T-Table 查表法	所有模式	高	4 KB 缓存占用
Shuffle 优化	—	负收益	额外 SIMD 开销
ShiftRows	—	—	—
AES-NI 硬件指令	所有模式	极高	需 CPU 支持 (主流)
AVX2 256 位聚合	CTR/GCM	中	需 AVX2 支持
8 路并行加密	CTR/GCM	高	寄存器压力大

关键结论：

1. **优化必须针对真正的性能瓶颈**：Shuffle 方案的经验教训表明，盲目应用 SIMD 可能适得其反。ShiftRows 在 AES 中开销占比极小，不值得单独优化。
2. **AES-NI 是最优方案**：对于支持 AES-NI 的 CPU，直接使用硬件指令可获得最佳性能，且代码简单可靠。
3. **并行度存在边际收益**：在 AES-NI 基础上从 4 路增加到 8 路，性能提升有限，内存带宽成为主要限制因素。
4. **模式选择影响显著**：XTS 模式因数据依赖无法有效并行，性能远低于 CTR/GCM，实际应用中应谨慎选择。

在实际应用中，**AES-NI 8 路并行**方案是最佳选择，兼容性好（几乎所有现代 x86 CPU 均支持），性能极佳。本实验结果为 AES 在通用处理器上的高性能实现提供了全面的参考数据，并揭示了优化策略选择的重要性。

5 GIFT-64 算法实现及优化

5.1 GIFT-64 算法简介

GIFT-64 是一种轻量级分组密码，由 Banik 等人于 2017 年提出。其分组长度为 64 位，密钥长度为 128 位（本实验采用），迭代轮数为 28 轮。GIFT-64 采用 **SPN（替代-置换网络）** 结构，每轮包含以下操作：

- **SubCells**：将状态分为 16 个 4 位半字节，每个半字节应用相同的 4×4 S 盒。

$$S(x) = \{1, 10, 4, 12, 6, 15, 3, 9, 2, 13, 11, 7, 5, 0, 8, 14\}$$

- **PermBits**: 对状态的 64 个比特进行固定置换, 提高扩散性。
- **AddRoundKey**: 将轮密钥 (64 位) 与状态异或。

密钥扩展使用简单的移位和常数更新, 每轮引入一个 4 位轮常数。

GIFT-64 的设计目标是硬件高效, 但在软件实现中, 其 4 位 S 盒和比特级置换 (PermBits) 构成了主要性能瓶颈。软件实现面临以下挑战:

1. 4 位 S 盒导致字节级查表效率低下 (每字节需处理两个半字节)。
2. PermBits 是比特级置换, 普通实现需逐位操作, 开销巨大。
3. 28 轮迭代带来大量循环和数据处理开销。

5.2 基础实现 (Base)

基础实现严格遵循算法定义, 采用逐半字节 S 盒查表和逐位 PermBits 置换。典型代码片段如下:

```
1 void gift64_encrypt_base(const uint8_t in[8], uint8_t out[8], const uint64_t rk[28]) {
2     uint64_t state = load64(in);
3     for (int r = 0; r < 28; r++) {
4         // SubCells: 逐半字节查表
5         uint64_t s = 0;
6         for (int i = 0; i < 16; i++)
7             s |= (uint64_t)GS[(state >> (4*i)) & 0xF] << (4*i);
8         // PermBits: 逐位查掩码并置位
9         uint64_t p = 0;
10        for (int i = 0; i < 64; i++)
11            if (s & masks[i]) p |= 1ULL << i;
12        state = p ^ rk[r];
13    }
14    store64(out, state);
15 }
```

此实现结构清晰, 但存在严重性能问题:

- 每轮需 16 次半字节查表 (GS 数组) 和 64 次位检查循环, 指令数庞大。
- 逐位操作无法利用 64 位寄存器的并行能力。
- 28 轮循环带来额外分支和跳转开销。

实验中 Base 版本的 CTR 模式吞吐率仅为约 1.35 MB/s, 验证了其极低的软件效率。

5.3 T-Table 优化

5.3.1 优化原理

GIFT-64 的 SubCells 和 PermBits 可以合并为预计算查表操作。核心思想是将 8 个输入字节 (64 位状态) 分别作为索引, 预先计算每个字节对最终 64 位置换输出的贡献 (掩码)。具体地:

- 将 64 位状态视为 8 个字节 $b_0, b_1, \dots, b_7$ 。
- 对于每个字节位置 i 和字节值 v (0-255), 预计算一个 64 位掩码 $P_i[v]$, 表示该字节经 S 盒替换和 PermBits 置换后对状态的贡献。
- 一轮变换可表示为:

$$state = \bigoplus_{i=0}^7 P_i[b_i] \oplus rk[r]$$

这样每轮只需 8 次 64 位查表和 8 次异或操作, 完全消除了半字节 S 盒和逐位置换的循环。

5.3.2 实现细节

本实验预计算了 8 个表, 每个表 256 个 64 位条目, 总大小为 $8 \times 256 \times 8 = 16$ KB, 可放入 L2 缓存。同时将 28 轮循环完全展开, 减少循环控制开销。

5.3.3 性能收益

T-Table 版本的 CTR 模式吞吐率达到 25.72 MB/s, 相比 Base 提升约 19 倍, 证明了预计算查表对 GIFT-64 这类比特置换密集的算法极为有效。

5.4 SIMD 向量化 S 盒尝试 (SSSE3 / AES-NI / AVX2)

由于 GIFT-64 的 S 盒为 4 位输入, 可用 ‘pshufb’ (SSSE3) 指令加速查表。同时, AES-NI 的 ‘aesencast’ 可生成 AES S 盒, 再通过 ‘pshufb’ 查表模拟 GIFT S 盒; AVX2 则利用 256 位操作扩展并行度。本实验分别实现了三种方案, 但加密核心仍为单块, 仅在模式层增加了并行路数。

5.4.1 SSSE3 方案: pshufb 查 S 盒

将 GIFT 的 4 位 S 盒扩展为 16 个 16 字节表(每个表对应高 4 位, 低 4 位为索引)。在每轮中, 将状态的 8 个字节拆分为高、低 4 位, 使用 ‘ $m_{shufblepi8}$ ’ GIFT S 4

5.4.2 AES-NI 方案: 仿射映射与 aesencast

利用 AES-NI 的 ‘ $m_{aesencast_{si128}}$ ’ AESS ‘pshufb’ GIFTS AES-NI SSSE3

5.4.3 AVX2 方案: 256 位 shuffle

使用 ‘ $m_{m256shufblepi8}$ ’ ‘ $m_{m256broadcastsi128_{si256}}$ ’ 256 64 256 8

5.5 工作模式实现优化

5.5.1 CTR 模式

CTR 模式天然支持并行加密不同计数器块。本实验实现:

- Base 和 T-Table 使用单块加密, 计数器逐一递增。

- SSSE3 和 AES-NI 使用 2 路并行 ('ctr₂x_ssse3' 'ctr₂x_aesni')
- AVX2 使用 8 路并行 ('ctr₈x_avx2' 256 XOR)

5.5.2 GCM 模式

GCM 的加密部分与 CTR 相同, 仅计数器初始值根据 IV 构建 (4 字节 IV + 4 字节固定字段)。本实验直接复用 CTR 并行函数, 因此性能与 CTR 基本一致。

5.5.3 XTS 模式

XTS 的 tweak 更新 ($GF(2^64)$) 2 XTS

5.6 实验结果

本次实验在 Intel Core i7-13700 (VMware, Ubuntu 20.04) 平台上运行, 测试数据量为 10 MB (因 Base 极慢, 减小规模), 每种方案重复 10 次取平均耗时, 并计算吞吐率。所有方案均通过正确性验证。终端输出截图如图 3 所示。

```
newl@ubuntu:~$ ./gift
```

GIFT-64 多方案 & 多模式 完整性能对比测试			
===== CTR 模式 (10MB × 10次) =====			
CTR	Base	74.270 秒	1.35 MB/s
CTR	T-Table	3.889 秒	25.72 MB/s
CTR	SSSE3	5.346 秒	18.71 MB/s
CTR	AES-NI	5.255 秒	19.03 MB/s
CTR	AVX2	5.307 秒	18.84 MB/s
===== GCM 模式 (10MB × 10次) =====			
GCM	Base	72.508 秒	1.38 MB/s
GCM	T-Table	3.842 秒	26.03 MB/s
GCM	SSSE3	5.238 秒	19.09 MB/s
GCM	AES-NI	5.269 秒	18.98 MB/s
GCM	AVX2	5.306 秒	18.85 MB/s
===== XTS 模式 (10MB × 10次) =====			
XTS	Base	74.245 秒	1.35 MB/s
XTS	T-Table	3.858 秒	25.92 MB/s
XTS	SSSE3	5.302 秒	18.86 MB/s
XTS	AES-NI	5.222 秒	19.15 MB/s
XTS	AVX2	5.275 秒	18.96 MB/s
===== 测试完成 =====			

图 3: GIFT-64 多方案多模式性能测试结果

性能数据汇总于表 5。

表 5: GIFT-64 各模式性能数据 (10MB $\times$ 10 次)

实现方案	CTR		GCM		XTS	
	耗时 (s)	吞吐 (MB/s)	耗时 (s)	吞吐 (MB/s)	耗时 (s)	吞吐 (MB/s)
Base	74.270	1.35	72.508	1.38	74.245	1.35
T-Table	3.889	25.72	3.842	26.03	3.858	25.92
SSSE3	5.346	18.71	5.238	19.09	5.302	18.86
AES-NI	5.255	19.03	5.269	18.98	5.222	19.15
AVX2	5.307	18.84	5.306	18.85	5.275	18.96

分析结果可见：

- **T-Table 效果显著：**吞吐率从 1.35 MB/s 提升至 25.72 MB/s，提升约 19 倍，是最有效的软件优化方法。
- **SIMD 方案性能相近但不及 T-Table：**SSSE3、AES-NI 和 AVX2 的吞吐率均在 18–19 MB/s 之间，比 T-Table 慢约 25%。原因在于：
 - GIFT 的 S 盒仅有 16 个条目，标量查表开销已很低，向量化带来的额外指令（如提取、广播）抵消了查表加速。
 - PermBits 仍是主要瓶颈，而 T-Table 已将置换完全预计算，SIMD 方案未能改进置换部分。
- **模式影响有限：**CTR、GCM、XTS 三者的性能差异很小，因为所有方案在 XTS 下均退化为单块加密，而并行优化在 XTS 中无效，但 T-Table 的单块性能已经足够好。

5.7 优化小结与展望

通过从 Base $\rightarrow$ T-Table 的优化，GIFT-64 的软件性能获得了质的飞跃，证明了预计算查表对于比特置换密集型算法的有效性。然而，SIMD 向量化 S 盒（SSSE3/AES-NI/AVX2）在 GIFT-64 上并未超越 T-Table，主要有以下原因：

- GIFT 的 4 位 S 盒本身很小，标量查表已经很快，SIMD 的额外指令开销抵消了并行收益。
- GIFT 的性能瓶颈在于 PermBits 而非 S 盒，而 T-Table 已完全消除置换开销，SIMD 方案未针对置换做改进。

PPT 中提到的 **fixslicing** 技术通过置换消除和超轮合并，可以进一步提升 GIFT 的性能，但本实验未实现。不同优化技术的适用性总结如表 6：

表 6: GIFT-64 优化技术对比

优化技术	适用场景	性能收益	代价
T-Table 预计算置换	所有模式	高 (19×)	16 KB 缓存占用
SSSE3 pshufb S 盒	—	中	额外向量指令
AES-NI 仿射映射	—	中	依赖 AES-NI
AVX2 256 位并行	CTR/GCM	中	需 AVX2 支持
Fixslicing (未实现)	所有模式	高	实现复杂

对于实际应用，**T-Table 方案**是 GIFT-64 软件实现的最佳选择，兼顾了性能和代码复杂度。而 SIMD 方案在此算法上收益有限，不推荐作为主要优化手段。本实验结果为轻量级密码 GIFT-64 在通用处理器上的实现提供了有价值的参考，同时也揭示了算法特性对优化策略选择的决定性影响。

6 TWINE 算法实现及优化

6.1 TWINE 算法简介

TWINE 是一种轻量级分组密码，由 Suzaki 等人于 2011 年提出，专为资源受限环境（如 RFID、传感器网络）设计。其分组长度为 64 位，密钥长度支持 80 位和 128 位两种版本，本实验采用 80 位密钥，迭代轮数为 36 轮。TWINE 采用 **广义 Feistel 结构** (Generalized Feistel Structure)，在硬件实现上具有较低的门电路开销，但在软件实现中面临独特的性能挑战。

每轮加密操作可形式化描述为：

- 将 64 位状态分为左右两个 32 位半块 L 和 R 。
- 右半部分 R 与轮密钥 RK_r 异或，得到 $F = R \oplus RK_r$ 。
- 将 F 拆分为 8 个 4 位半字节 $f_0, f_1, \dots, f_7$ ，其中 $f_i = (F \gg (4i)) \& 0xF$ 。
- 每个半字节经过同一个 4×4 S 盒：

$$S(x) = \{0xc, 0x0, 0xf, 0xa, 0x2, 0xb, 0x9, 0x5, 0x8, 0x3, 0xd, 0x7, 0x1, 0xe, 0x6, 0x4\}$$

该 S 盒具有优良的差分和线性特性，适合轻量级实现。

- 将 S 盒输出的 8 个半字节按固定置换 $SHUFFLE = \{5, 0, 1, 4, 7, 12, 3, 8\}$ 重新排列到右半部分的 8 个 4 位位置。注意， $SHUFFLE[5]=12$ 和 $SHUFFLE[7]=8$ 超出 32 位范围（最大位置为 7），因此这两个位置的输出被丢弃。
- 新右半块 R' 与左半块 L 异或，得到新左半块 $L' = L \oplus R'$ ；原右半块 R 直接成为新右半块。

TWINE 的密钥扩展算法相对简单：将 80 位密钥拆分为 20 个 4 位半字节，每轮从密钥数组中取 8 个半字节构成 32 位轮密钥，并引入 6 位轮常数，然后对密钥数组进行循环移位。这种设计适合硬件实现，但软件中频繁的 4 位操作增加了处理开销。

TWINE 的软件实现面临以下核心挑战：

1. 4 位 S 盒导致字节级查表效率低下——每字节包含两个半字节，需分别处理，增加了查表次数和位操作复杂度。
2. 半字节置换 SHUFFLE 中的无效位置（12 和 8）迫使软件实现需进行条件判断或特殊处理，破坏了指令流的连续性。
3. 36 轮迭代带来约 576 次 S 盒查表和 288 次半字节移位/异或操作，累积开销显著。
4. 80 位密钥的非标准长度导致密钥装载和更新需要额外的位操作。

6.2 基础实现 (Base)

基础实现严格遵循算法定义，采用逐半字节 S 盒查表和条件判断。典型代码片段如下：

```
1 void twine_encrypt_base(const uint8_t in[8], uint8_t out[8], const uint64_t rk[ROUNDS]) {
2     uint64_t X = 0;
3     for (int i = 0; i < 8; i++) X |= ((uint64_t)in[i]) << (8 * i);
4     for (int r = 0; r < ROUNDS; r++) {
5         uint32_t L = (uint32_t)(X >> 32), R = (uint32_t)X;
6         uint32_t F = R ^ (uint32_t)rk[r];
7         uint32_t new_R = 0;
8         if (SHUFFLE[0] < 8) new_R |= ((uint32_t)SBOX[F & 0xF]) << (4 * SHUFFLE[0]);
9         if (SHUFFLE[1] < 8) new_R |= ((uint32_t)SBOX[(F >> 4) & 0xF]) << (4 * SHUFFLE[1]);
10        // 处理 SHUFFLE[2] 到 SHUFFLE[6] 类似，仅 SHUFFLE[5]=12 和 SHUFFLE[7]=8 被跳过
11        X = ((uint64_t)R << 32) | (L ^ new_R);
12    }
13    for (int i = 0; i < 8; i++) out[i] = (uint8_t)(X >> (8*i));
14 }
```

此实现结构清晰，但存在以下性能瓶颈：

- 每轮需 6 次 S 盒查表和移位操作（跳过 SHUFFLE[5]=12 和 SHUFFLE[7]=8 两个无效位置），36 轮共 216 次查表，每次查表都伴随条件判断。
- 条件判断（‘if’）引入分支预测风险，在现代超标量处理器上可能造成流水线停顿。
- 状态变量 X 在每轮中拆分为 L 和 R ，并重新组合，增加了数据移动开销。
- 36 轮循环控制（递增、比较、跳转）累积约 36 次额外指令。

实验中 Base 版本的 CTR 模式吞吐率约为 60.98 MB/s，作为后续优化的参考基准。

6.3 T-Table 优化

6.3.1 优化原理与表构造

T-Table 优化的核心思想是将 S 盒和半字节置换合并为预计算查找表，以空间换时间。对于输入 F （32 位字）的 8 个 4 位半字节，预先计算每个半字节对最终 new_R 的贡献（32 位掩码）。定义 8 个表 $TT[pos][v]$ ，其中 pos 为半字节位置（0-7）， v 为 S 盒输入（0-15），输出为：

$$TT[pos][v] = \begin{cases} S[v] \ll (4 \cdot SHUFFLE[pos]), & SHUFFLE[pos] < 8 \\ 0, & \text{否则} \end{cases}$$

每个表大小为 $16 \times 4 = 64$ 字节，8 个表共计 512 字节，可完全放入 L1 缓存。

一轮变换可简化为 8 次查表和 7 次异或操作：

```
1 uint32_t new_R = TT[0][F & 0xF] ^ TT[1][(F>>4)&0xF] ^  
2     TT[2][(F>>8)&0xF] ^ TT[3][(F>>12)&0xF] ^  
3     TT[4][(F>>16)&0xF] ^ TT[5][(F>>20)&0xF] ^  
4     TT[6][(F>>24)&0xF] ^ TT[7][(F>>28)&0xF];
```

相比 Base，T-Table 消除了所有条件分支，将 6 次条件查表简化为 8 次无条件查表。

6.3.2 性能结果与深入分析

实验结果显示，T-Table 方案的 CTR 吞吐率仅为 58.54 MB/s，甚至低于 Base (60.98 MB/s)。这一反直觉的结果需要深入分析：

- **查表次数比较：**Base 每轮实际执行 6 次 S 盒查表（跳过两个无效位置），而 T-Table 执行 8 次查表——即使某些表值为 0。多出的 2 次查表（每次需内存加载和异或）增加了约 25% 的查表开销。
- **内存访问模式：**Base 的 S 盒仅 16 字节，始终在 L1 缓存中，访问延迟极低；T-Table 的 8 个表共 512 字节，虽也在 L1 中，但访问模式更分散，可能轻微增加缓存冲突。
- **无效位置的特殊性：**SHUFFLE[5]=12 和 SHUFFLE[7]=8 对应的表值恒为 0，但硬件仍需执行加载、异或操作，无法被编译器优化掉。
- **指令级并行：**Base 的 6 次查表因条件判断可能串行执行；T-Table 的 8 次查表虽无分支，但查表间存在数据依赖（异或链），限制了乱序执行的并行度。

这一结果说明：**优化并非简单的“查表替换”**，必须精确权衡操作数变化、内存访问模式和数据依赖关系。对于 TWINE，Base 实现已充分利用了算法的“稀疏置换”特性，T-Table 反而引入了冗余操作。

6.4 SSSE3 向量化 S 盒优化

6.4.1 优化原理与实现技术

SSSE3 方案的核心是利用 ‘ $mshuf fle_{\pi 8}$ ’ $pshufb$ 4 S $pshufb$ SSSE3 16 4 16 16 实现分为以下几个步骤：

1. **表预计算：**对于每个高半字节值 $hi \in [0, 15]$ ，构造一个 16 字节表 T_{hi} ，使得 $T_{hi}[lo] = S[(hi \ll 4) | lo]$ 。共 16 个表，每个 16 字节，总计 256 字节。
2. **数据加载与拆分：**将 32 位 F 加载到 XMM 寄存器，拆分为高 4 位和低 4 位两个向量。
3. **表选择与查表：**对每个字节位置（共 4 个字节，对应 8 个半字节），使用高 4 位选择对应的表，再用低 4 位作为 $pshufb$ 索引进行查表。
4. **掩码组合：**由于 $pshufb$ 一次只能查一个表，而不同字节的高 4 位可能不同，需要通过 ‘ $mcmpeq_{\pi 32}$ ’

代码实现中，4 次迭代处理 4 个字节，每次处理 2 个半字节（一个字节），利用了 SIMD 的 16 字节并行能力。

6.4.2 性能收益与原因分析

SSSE3 方案的 CTR 吞吐率达到 85.10 MB/s，相比 Base 提升约 39.5%，相比 T-Table 提升约 45.4%。性能提升的来源包括：

- **指令级并行**：pshufb 单条指令完成 16 个查表操作，大幅减少了指令数。
- **消除分支**：向量化实现完全消除了 Base 中的条件判断和 T-Table 中不必要的查表。
- **寄存器重用**：16 个 S 盒表保持在 XMM 寄存器中（或 L1 缓存），查表速度极快。
- **数据并行**：虽然 TWINE 是 32 位数据，但 SIMD 指令同时处理多个字节，提高了吞吐。

6.5 AVX2 256 位并行尝试

6.5.1 优化原理与实现

AVX2 方案将 SSSE3 的 128 位操作扩展至 256 位，主要使用以下指令：

- `'m256_shuffle_pi8' 256 pshufb`
- `'m256_broadcastsi128_si256' 128 256 128`
- `'m256_mpeqi32' 'm256_and_si256'`

理论上，AVX2 可同时处理两倍于 SSSE3 的数据量，预期性能更高。

6.5.2 性能结果与深入分析

AVX2 方案的 CTR 吞吐率为 74.05 MB/s，虽优于 Base (60.98 MB/s)，但低于 SSSE3 (85.10 MB/s) 和 BMI2 (88.49 MB/s)。深入分析原因：

- **数据宽度不匹配**：TWINE 每轮仅处理 32 位数据（4 个字节），而 AVX2 寄存器为 256 位（32 字节），利用率仅为 12.5%。大量寄存器位被浪费，但指令延迟却比 128 位操作更高。
- **广播开销**：`'m256_broadcastsi128_si256' 128`
- **通道间操作受限**：AVX2 的 shuffle 指令在 128 位通道内操作，跨通道数据移动需要额外指令（如 `'m256_permute4x64_epi64' 32`）
- **寄存器压力**：AVX2 的 16 个 YMM 寄存器虽多，但 256 位操作占用的寄存器资源更多，可能导致寄存器溢出。
- **功耗与温度**：256 位操作通常比 128 位操作功耗更高，可能导致 CPU 降频，影响峰值性能。

此案例表明：**选择 SIMD 宽度需与数据粒度匹配**。对于 TWINE 这类小分组算法，128 位 SIMD 可能是最优选择，盲目使用更宽向量可能适得其反。

6.6 BMI2 比特置换优化

6.6.1 优化原理与指令分析

BMI2 (Bit Manipulation Instructions 2) 是 Intel Haswell 及后续微架构引入的位操作扩展。其中 $\text{'pdep}_u32'$ 和 TWINE

TWINE 的 SHUFFLE 可视为一个分散操作：将 S 盒输出的 8 个 4 位值（共 32 位）分散到 32 位目标中的 8 个 4 位位置（位置 5、0、1、4、7、12、3、8，其中位置 12 和 8 超出范围被丢弃）。但 $\text{'pdep}_u32'$ 只能操作 4 位。

实现步骤：

1. 将 32 位 F 拆分为 8 个半字节，分别查 S 盒。
2. 对每个 S 盒输出 (4 位), 使用 $\text{'pdep}_u32'$ 将 4 位值左移 $\text{SHUFFLE}[pos]*4$ 位，得到 $\text{'SHUFFLE}[pos] < 8'$ 。
2. 对 $\text{SHUFFLE}[pos] < 8$ 的 6 个位置执行 $\text{'pdep}_u32'$ ，将 4 位值左移 12 8 位。
2. 将所有结果异或，得到最终的 $\text{'new}_R'$ 。

实际实现中，由于 $\text{'pdep}_u32'$ 只能操作 4 位，S 盒输出 32 位。

6.6.2 性能收益与原因分析

BMI2 方案在 CTR 模式下达到 88.49 MB/s，为所有方案中最高，相比 Base 提升约 45.1%，相比 SSSE3 提升约 4.0%。性能优势源于：

- 指令级效率：使用 $\text{'pdep}_u32'$ 指令。
- 数据依赖最短：每个半字节的 $\text{'pdep}_u32'$ 指令。
- 无内存访问：使用 $\text{'pdep}_u32'$ 指令，无需 SSSE3 pshufb 指令，L1 缓存。
- 代码紧凑：BMI2 方案的核心循环指令数最少，有利于指令缓存和取指效率。

6.7 工作模式实现优化

6.7.1 CTR 模式

CTR 模式天然支持并行加密不同计数器块，本实验充分利用了这一特性：

- **Base 与 T-Table**：采用单块加密，计数器逐一递增。每加密一块需执行完整的 36 轮加密，无并行。
- **SSSE3**：实现 4 路并行（使用 $\text{'twine}_{ctr_sse3_4x}'$ 指令，128 XMM 寄存器，4 路并行，4 个 128 位块）。
- **AVX2**：实现 8 路并行（使用 $\text{'twine}_{ctr_avx2_8x}'$ 指令，256 YMM 寄存器，8 路并行，4 个 256 位块）。
- **BMI2**：同样实现 8 路并行，使用 4 组双路 BMI2 加密（使用 $\text{'twine}_{ncrypt_bmi2_2x}'$ 指令，AVX2 256 XOR 指令）。

各并行方案的计数器更新均采用批量增量（4 或 8），避免了每块单独递增的开销。

6.7.2 GCM 模式

GCM 的加密部分与 CTR 完全相同，唯一区别在于初始计数器的构建：12 字节 IV + 4 字节固定字段（初始值为 2）。本实验直接复用 CTR 并行函数，因此 GCM 模式下各方案性能与 CTR 基本一致（表 7 中 GCM 与 CTR 数据高度吻合）。

6.7.3 XTS 模式

XTS 模式为每个数据块生成独立的 tweak 值，且 tweak 在 $GF(2^64)$ 中 $tweak_{i+1} = tweak_i \cdot 2$ ，无法像 CTR 那样预计算多个独立计数器并行加密。本实验在 XTS 模式下，所有方案均采用双路并行加密两个连续块（因 tweak 可预计算两个连续值），但无法扩展到 4 路或 8 路。

XTS 模式下的性能表现：

- **Base**: 59.68 MB/s，与 CTR 的 60.98 MB/s 接近，因 Base 本身无双路并行，模式差异影响较小。
- **SSSE3**: 76.45 MB/s，低于 CTR 的 85.10 MB/s（约 10.2% 下降），因双路并行相比 4 路并行减半。
- **AVX2**: 68.37 MB/s，低于 CTR 的 74.05 MB/s（约 7.7% 下降），同样因并行度受限。
- **BMI2**: 77.23 MB/s，仍为 XTS 模式最高，但低于 CTR 的 88.49 MB/s（约 12.7% 下降）。

XTS 的结果清晰展示了**并行度与算法数据依赖的强关联**：CTR/GCM 因无数据依赖可充分并行，XTS 因 tweak 依赖严重限制了并行优化空间。

6.8 实验结果

本次实验在 Intel Core i7-13700 (VMware, Ubuntu 20.04) 平台上运行，测试数据量为 100 MB，每种方案重复 10 次取平均耗时并计算吞吐率。所有方案均通过正确性验证（与 Base 密文一致）。终端输出截图如图 4 所示。

```
newl@ubuntu:~$ ./twine
```

TWINE 多方案 & 多模式 性能对比测试					
===== CTR 模式 (100MB × 10次) =====					
CTR	Base		16.399 秒		60.98 MB/s
CTR	T-Table		17.083 秒		58.54 MB/s
CTR	SSSE3		11.751 秒		85.10 MB/s
CTR	AVX2		13.505 秒		74.05 MB/s
CTR	BMI2		11.301 秒		88.49 MB/s
===== GCM 模式 (100MB × 10次) =====					
GCM	Base		15.889 秒		62.94 MB/s
GCM	T-Table		16.977 秒		58.90 MB/s
GCM	SSSE3		11.686 秒		85.57 MB/s
GCM	AVX2		13.378 秒		74.75 MB/s
GCM	BMI2		11.426 秒		87.52 MB/s
===== XTS 模式 (100MB × 10次) =====					
XTS	Base		16.755 秒		59.68 MB/s
XTS	T-Table		17.424 秒		57.39 MB/s
XTS	SSSE3		13.080 秒		76.45 MB/s
XTS	AVX2		14.625 秒		68.37 MB/s
XTS	BMI2		12.949 秒		77.23 MB/s
===== 测试完成 =====					

图 4: TWINE 多方案多模式性能测试结果

性能数据汇总于表 7。

表 7: TWINE 各模式性能数据 (100MB × 10 次)

实现方案	CTR		GCM		XTS	
	耗时 (s)	吞吐 (MB/s)	耗时 (s)	吞吐 (MB/s)	耗时 (s)	吞吐 (MB/s)
Base	16.399	60.98	15.889	62.94	16.755	59.68
T-Table	17.083	58.54	16.977	58.90	17.424	57.39
SSSE3	11.751	85.10	11.686	85.57	13.080	76.45
AVX2	13.505	74.05	13.378	74.75	14.625	68.37
BMI2	11.301	88.49	11.426	87.52	12.949	77.23

对实验结果的综合分析与讨论：

- **优化效果排序：**BMI2 (88.49 MB/s) > SSSE3 (85.10 MB/s) > AVX2 (74.05 MB/s) > Base (60.98 MB/s) > T-Table (58.54 MB/s)。排序清晰反映了各优化手段的实际有效性。
- **AVX2 的意外表现：**作为最宽的 SIMD 方案，AVX2 性能反而不及 SSSE3，这打破了“更宽 SIMD 一定更快”的常见误解，对后续轻量级密码的 SIMD 优化具有警示意义。
- **T-Table 的失效：**这是本实验最重要的教训之一——查表优化并不总是有效，需要仔细分析算法的“稀疏性”和“空位”特征。对于 TWINE 这类存在大量无效位置的算法，保持轻量级

条件判断可能优于预计算查表。

- **模式差异**：CTR 和 GCM 性能高度一致（偏差 $< 2\%$ ），验证了 GCM 加密部分与 CTR 等价的事实；XTS 模式因数据依赖，所有方案性能均下降 10–20%。
- **绝对吞吐量**：所有方案的吞吐率均在 60–90 MB/s 范围内，相比 AES 或 SM4 的 GB/s 级别，反映了 TWINE 作为轻量级密码在通用软件实现中的效率上限——其设计目标本就是硬件而非软件高效。

6.9 优化小结与启示

通过 TWINE 的五种实现方案对比，我们获得了一系列有价值的发现：

表 8: TWINE 优化技术综合对比

优化技术	适用场景	性能收益	代价/限制
T-Table 查表法	一般密码算法	负收益	额外查表开销，不适合稀疏置换
SSSE3 pshufb	半字节 S 盒	高 (+39.5%)	需 SSSE3 支持，128 位操作
AVX2 256 位	大数据块处理	中 (+21.4%)	数据宽度与算法粒度不匹配
BMI2 pdep	比特/半字节置换	最高 (+45.1%)	仅限 Haswell+ CPU 支持
循环展开	固定轮数	中	代码膨胀，可能影响指令缓存
多路并行 (4/8 路)	CTR/GCM	高	对 XTS 模式无效

核心结论与设计启示：

1. **优化必须基于算法微观结构**：TWINE 的 SHUFFLE 包含无效位置，使得 T-Table 这种“万能”优化方法失效。优化前必须深入理解算法的数据流和操作特征。
2. **SIMD 宽度不是越大越好**：AVX2 在 TWINE 上表现不及 SSSE3，说明 SIMD 宽度应与数据粒度匹配。对于 32 位或 64 位分组，128 位 SIMD 可能是最优选择；256 位或 512 位反而引入额外开销。
3. **专用指令的价值**：BMI2 的 ‘ $pdep_{u32}$ ’ *CPU* *BMI2* *AVX-512* *GFNI*
3. **模式选择影响优化策略**：CTR/GCM 因无数据依赖，可充分利用 SIMD 并行；XTS 因 tweak 依赖限制了并行度，优化重点应放在单块加密效率而非并行路数上。

实际应用建议：

- 若目标 CPU 支持 BMI2（Intel Haswell 及后续），**BMI2 方案**是 TWINE 的最优实现，兼具最高性能和简洁的代码结构。

- 若需兼容更广泛的 x86-64 CPU（支持 SSSE3 但不一定支持 BMI2），**SSSE3 方案**是最佳选择，性能接近 BMI2（差距约 4%），兼容性更广。
- 应避免使用 **T-Table 方案**，其负收益表明查表法并非普适优化。
- 对于 AVX2，建议仅当处理更宽数据（如 256 位分组或批量处理多个 64 位分组）时考虑使用，而非单块加密。

本实验结果为轻量级密码 TWINE 在通用处理器上的高性能实现提供了系统的参考，同时也为其他 Feistel 结构轻量级密码的优化策略选择提供了方法论借鉴。不同算法对优化技术的响应差异，强调了性能工程需要算法与微架构双重视角的结合。

7 总结

至此，本实验对四种典型分组密码算法——**AES-128**（标准 SPN 结构）、**SM4**（国标 Feistel 结构）、**GIFT-64**（轻量级 SPN）和 **TWINE**（轻量级 Feistel 结构）——在 x86-64 平台上进行了系统的软件实现与优化研究。每种算法均从基础实现出发，逐步应用 T-Table 查表法、SIMD 向量化（SSSE3/AVX2）、硬件指令（AES-NI/BMI2）及多路并行等优化手段，并在 CTR、GCM、XTS 三种常用工作模式下完成了完整的性能测试。以下从宏观角度总结各算法的优化效果、关键发现与共性启示。

7.1 各算法最优方案及加速比汇总

表 9 汇总了四种算法在 CTR 模式下的最优实现及其相对基础实现的加速比（吞吐率比值）。

表 9: 四种算法 CTR 模式最优性能与加速比

算法	基础吞吐 (MB/s)	最优方案	最优吞吐 (MB/s)	加速比
AES-128	77.92	AES-NI (8 路并行)	3558.17	45.7×
SM4	117.29	SSSE3 / AES-NI / AVX2	≈ 204	1.74×
GIFT-64	1.35	T-Table	25.72	19.1×
TWINE	60.98	BMI2 (8 路并行)	88.49	1.45×

可以看出，优化效果因算法结构差异极大：AES 因存在专用硬件指令（AES-NI）获得了数量级的提升；GIFT 因基础实现极为低效（逐位置换），T-Table 的预计算带来巨大收益；而 SM4 和 TWINE 的基础实现已相对高效，优化空间有限。

7.2 优化技术的适用性分析

不同优化技术在各算法上的表现差异显著，表 10 总结了主要优化手段的适用性。

表 10: 主要优化技术在不同算法上的适用性

优化技术	AES-128	SM4	GIFT-64	TWINE
T-Table 查表	有效 (5.2×)	有效 (1.58×)	极 有 效 (19×)	负收益
SSSE3 pshufb	未采用	有效 (1.75×)	部分有效	有效 (1.4×)
AES-NI 硬件指令	极 有 效 (45×)	中等 (1.74×)	部分有效	—
AVX2 256 位	有效 (1.03×)	有效 (1.74×)	部分有效	负收益
BMI2 pdep	—	—	—	最 有 效 (1.45×)
多路并行 (4/8 路)	有效	有效	有效	部 分 有 效 (XTS 受限)

关键发现：

- **AES-NI 是 AES 的专属优势**，对于其他算法只能通过仿射映射间接使用，收益有限。
- **T-Table 的成败取决于置换的稀疏性**：GIFT 的 PermBits 无空位，T-Table 效果极佳；TWINE 的 SHUFFLE 有空位，T-Table 反而增加开销。
- **AVX2 并非万能**：在 SM4 和 AES 上因数据宽度匹配（128 位分组）略有效果，但在 TWINE（64 位分组）上因宽度不匹配出现负收益。
- **专用指令（BMI2）对特定操作（比特置换）有奇效**，但受限于 CPU 支持。

7.3 工作模式对性能的影响

三种工作模式对各算法的性能影响呈现一致的规律：

- **CTR 与 GCM 性能基本持平**：因为 GCM 的加密部分与 CTR 相同，仅初始化计数器不同，两者吞吐率偏差通常在 2% 以内。
- **XTS 模式显著降低并行收益**：由于 tweak 的串行依赖 $(GF(2^m))^2$ ， $XTS \ll CTR/GCM$

因此，在实际应用中，若需高吞吐且可容忍认证开销，优先选用 GCM；若仅需加密且可并行，CTR 是最佳选择；XTS 适用于磁盘加密等场景，但需接受性能折中。

7.4 整体结论与展望

通过本次实验，我们得出以下总体结论：

1. **优化必须“因算法而异”**：不存在一种普适的优化方法。AES 依赖硬件指令，GIFT 依赖查表，TWINE 依赖专用位操作，SM4 则需平衡 T-Table 与 SIMD。盲目套用优化技术可能适得其反（如 TWINE 的 T-Table、AVX2）。

2. **指令集选择与数据宽度匹配至关重要**: AES-NI 和 BMI2 等专用指令效果远优于通用 SIMD; SIMD 宽度应与分组大小匹配, 128 位对 128 位分组 (AES/SM4) 效果最好, 256 位对 64 位分组 (TWINE) 反而引入额外开销。
3. **基础实现的质量影响优化天花板**: GIFT 基础实现极慢 (1.35 MB/s), 优化后提升 19 倍; 而 SM4 和 TWINE 的基础实现已较高效, 进一步优化空间有限 (< 2 倍)。
4. **工作模式的选择对性能有决定性影响**: CTR/GCM 的并行性使得 SIMD 优化充分受益, XTS 的串行依赖则大幅削弱并行优化的价值。

对实际应用的建议:

- 对于支持 AES-NI 的 CPU, 应优先使用 AES-NI 并行模式 (如 8 路 CTR), 可获得数十 GB/s 的吞吐。
- 对于 SM4, 若 CPU 支持 SSSE3, 推荐使用 SSSE3 向量化 T-Table 方案; 若追求最大兼容性, 使用标量 T-Table 已足够。
- 对于 GIFT-64, T-Table 方案是绝对首选, 性能远超其他方法。
- 对于 TWINE, 若 CPU 支持 BMI2, 应优先采用 BMI2 方案; 否则使用 SSSE3 方案。

本实验不仅为四种密码算法提供了高性能实现参考, 更揭示了算法结构、指令集特性和工作模式之间的深层交互关系, 为未来密码算法优化工作提供了方法论指导。随着 CPU 指令集的持续演进 (如 AVX-512、GFNI、ARM SVE2), 轻量级密码在通用处理器上的软件性能仍有进一步挖掘的空间。